

ETHENA · SECURITY REVIEW

Ethena

Pay Review.

A security review of Pay Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Ethena	Pay Review	Guardian	March 9 – April 8, 2026	April 15, 2026

§ 00

Contents.

46 pages — overview, confidence ranking, scope, 34 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 32 findings	P. 10
	Remediation Review · 2 findings	P. 43
§ 05	Disclaimer	P. 45
§ 06	About Guardian	P. 46

§ 01

Overview.

Guardian reviewed Pay Review across 2 review rounds. The final report includes 3 Medium, 7 Low, 24 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	chainnodesorg/ethenapay-wallet-contracts
CLIENT	Ethena	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review
PROJECT	Pay Review	MAIN REVIEW COMMIT	9122473f5310
PREPARED BY	Guardian	REMEDATION REVIEW COMMIT	498c18fb29c0
AUDIT WINDOW	March 9 – April 8, 2026		
FINAL REPORT	April 15, 2026		

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	3	0	2	0	1
Low	7	0	2	0	5
Informational	24	0	12	0	12
All severities	34	0	16	0	18

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Pay Review in `chainnodesorg/ethenapay-wallet-contracts`. The scope includes 2,159 source lines across 14 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/UserProxyFactoryBeacon.sol	319	643
src/WalletDiamond.sol	72	121
src/libraries/LibAllowance.sol	84	148
src/libraries/LibAuth.sol	673	1,357
src/libraries/LibDirectMode.sol	145	308
src/libraries/LibEIP712.sol	28	49
src/libraries/LibFactory.sol	22	44
src/libraries/LibRecovery.sol	108	215
src/libraries/LibReentrancyGuard.sol	25	50
src/facets/AllowanceFacet.sol	75	153
src/facets/AuthFacet.sol	266	578
src/facets/DirectModeFacet.sol	59	158
src/facets/RecoveryFacet.sol	157	299
src/facets/WithdrawalFacet.sol	126	251

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Compromised key can steal assets in direct mode	Trust Assumptions	MEDIUM	ACKNOWLEDGED	p. 10
M-02	Per-Signer Nonce Cross-Signer Replay	Signatures	MEDIUM	RESOLVED	p. 11
M-03	Compromised executor can instantly drain wallets	Trust Assumptions	MEDIUM	ACKNOWLEDGED	p. 12
L-01	Max signer cap blocks replacement flow	Unexpected Behavior	LOW	RESOLVED	p. 14
L-02	Passkey Removal Signs Mutable Index	Logical Error	LOW	RESOLVED	p. 15
L-03	Stale Timestamps After Direct Mode Transitions	Unexpected Behavior	LOW	RESOLVED	p. 16
L-04	Direct Mode Cancel Lacks State Binding	Logical Error	LOW	RESOLVED	p. 17
L-05	Wallet recovery jeopardized by factory owner	Trust Assumptions	LOW	ACKNOWLEDGED	p. 18
L-06	No path back to default recovery admins	Configuration	LOW	RESOLVED	p. 19
I-01	Inconsistent function naming	Best Practices	INFORMATIONAL	RESOLVED	p. 20
I-02	REENTRANCY_SLOT deviates from ERC-7201	Best Practices	INFORMATIONAL	RESOLVED	p. 21
I-03	Redundant exists checks before event emission	Informational	INFORMATIONAL	RESOLVED	p. 22
I-04	Wallet deploy event omits passkey data	Informational	INFORMATIONAL	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-05	No Per-Token Zero-Spend limit	Suggestion	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-06	Passkey Removal Event Omits Key Data	Events	INFORMATIONAL	RESOLVED	p. 25
I-07	Default recovery admins cannot be empty	Configuration	INFORMATIONAL	RESOLVED	p. 26
I-08	Recovery Config Events Lack Admin Data	Events	INFORMATIONAL	RESOLVED	p. 27
I-09	Unlimited mode skips daily spend accounting	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 28
I-10	Natspec says wallet nonce for allowance	Documentation	INFORMATIONAL	RESOLVED	p. 29
I-11	Max amount transfer handling	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 30
I-12	Executor Address Can Be Set As Recovery Admin	Validation	INFORMATIONAL	RESOLVED	p. 31
I-13	Allowance Set During Pause Cannot Be Spent	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 32
I-14	Settlement Flows Lack Batch Execution	Suggestion	INFORMATIONAL	ACKNOWLEDGED	p. 33
I-15	Executor settlement depends on chain state	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 34
I-16	ETH naming for Avalanche deployment	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 35
I-17	Pending Executor Can Be Added As Wallet Owner	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 36
I-18	Facet Upgrade Can Break Pending Recovery	Upgradeability	INFORMATIONAL	ACKNOWLEDGED	p. 37
I-19	Recovery Admin Validation Copies Calldata	Gas Optimization	INFORMATIONAL	RESOLVED	p. 38
I-20	Executor Verification Copies Calldata To Memory	Gas Optimization	INFORMATIONAL	RESOLVED	p. 39
I-21	Ownership Transfer Lacks Timelock	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 40

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-22	Natspec omits self-address check	Documentation	INFORMATIONAL	RESOLVED	p. 41
I-23	Skipped WebAuthn verification steps	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 42

REMEDIATION REVIEW

L-01	Default recovery path doesn't modify optedOut	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 43
I-01	Lack of signer parameter documentation	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 44

MEDIUM **M-01**

ACKNOWLEDGED

Compromised key can steal assets in direct mode

CATEGORY Trust Assumptions
LOCATION src/facet/DirectModeFacet.sol
REVIEW Main Review

DESCRIPTION

When direct mode is active, `directEthWithdrawal()` and `directTokenWithdrawal()` require a single signer via `LibAuth.verifyOwnerSignature()`. If any one signer is compromised, the attacker can drain funds without executor co-signing. The 1-hour activation delay only helps if direct mode is not activated.

All actions that help the user - removing owner, exiting direct mode - can be cancelled by the compromised signer. This leaves the real owners of the wallet with only one possible mitigation - race with the malicious actor to withdraw all funds, which in most of the time results in the malicious actor taking the assets.

This issue is also present when direct mode is not turned on because the executor will most probably still sign a withdrawal message and let the compromised key withdraw funds.

RECOMMENDATION

Consider requiring `min(2, totalSignerCount())` of the owners to have signed when funds are withdrawn in direct mode.

One downside of this solution is that wallets that want to enter direct mode will be unable to be used for the next 24 hours if there is more than one signer, but the key for only one of them is usable.

Also rethink if the flow when direct mode is not turned on needs to be reworked, or additional authentication means need to be added to the executor sign path.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Per-Signer Nonce Cross-Signer Replay

CATEGORY Signatures
LOCATION global
REVIEW Main Review

DESCRIPTION

The wallet stores replay-protection nonces per signer, but the EIP-712 payloads do not include the signer identity whose nonce is being used. Instead, the contract hashes only the action parameters together with a bare nonce and deadline, then recovers the signer from the submitted signature and consumes

`signerNonces[signerKey]`. This affects all owner signature types supported by the wallet, including ECDSA, EIP-1271, and WebAuthn, because the signer is selected only after verifying the same digest.

This creates a mismatch between what is actually being authorized and which replay-protection state gets consumed. The same payload can be authorized by any registered signer, and the contract will accept it as long as that signer's current nonce matches the provided numeric nonce. A malicious signer can therefore advance their own nonce until it matches another signer's nonce, then submit the same action payload using their own valid signature.

This is especially dangerous for withdrawal flows. In executor withdrawal paths, the same executor signature can also be reused, since the executor signs the same digest and that digest is not tied to a specific owner signer. As a result, a single withdrawal authorization payload can be reused across multiple signer nonce lanes instead of being single-use for the wallet.

RECOMMENDATION

Bind the signer's identity to every owner-signed EIP-712 message that uses per-signer nonces, and check that the recovered signer matches the one included in the signed data. For actions that should only be usable once for the whole wallet, such as withdrawals, consider using a single global wallet nonce.

RESOLUTION

Status: Resolved.

MEDIUM **M-03**

ACKNOWLEDGED

Compromised executor can instantly drain wallets

CATEGORY Trust Assumptions
LOCATION AllowanceFacet.sol
REVIEW Main Review

DESCRIPTION

The executor has a daily limit of tokens it can transfer on behalf of the wallet by calling `spendDailyAllowance()` and `spendDailyAllowanceEth()`. It's expected for most wallets to have given the executor full allowance because the default value `0` is interpreted as unlimited spending.

The case where the executor is compromised can be countered by allowing the user to opt out of the executor, which can be done by entering direct mode. However, there is a 1 hour window that must pass before the direct mode takes effect. The allowance for the executor becomes irrevocable because `instantSetAllowance()` requires a signature both from a valid owner and the executor itself.

When the protocol team notices the executor is compromised, they can call `UserProxyFactoryBeacon.globalPause()` and block the spending, but this may not be sufficient, because the compromised executor can atomically drain all wallets that have given permission to it.

RECOMMENDATION

Consider implementing a two step spending mechanism, where the executor makes a request and the actual funds transfer happens after a small delay, for example 5 minutes.

In this period between request and execution, the protocol team has enough time to call `globalPause()` and schedule executor change. The payment processing should revert if the factory is paused or the current executor is not the one that initiated the request. The downside of this approach is that any unprocessed legitimate payments that were initiated in the waiting period before the protocol was paused, will probably be lost. It's actually the same right now, but with a smaller time period.

Notice that in case the factory owner goes rogue as well, there is nothing that the user can do to prevent their funds from being stolen.

M-03 Compromised executor can instantly drain wallets

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-01**

RESOLVED

Max signer cap blocks replacement flow

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAuth.sol#L918 , https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAuth.sol#L628
REVIEW	Main Review

DESCRIPTION

`LibAuth.initiatePendingPasskey()` and `LibAuth.initiateAddOwner()` revert when the current signer count hits `MAX_PASSKEYS / MAX_OWNERS`. This check blocks signer replacement for additional 24 hours when the length of the current array is at the max.

For example:

- There are 10 passkeys
- `Passkey A` should be replaced with `Passkey B`
- The admins of the wallet cannot initiate both the addition and the removal at the same time. They have to wait 24 hours for `Passkey A` to be removed and then wait another 24 hours to add `Passkey B`.

RECOMMENDATION

Because the same cap check is already present in `executePendingPasskey()/executeAddOwner()`, you can either remove it from `initiatePendingPasskey()/initiateAddOwner()` or relax it to being applied only if the pending removals are empty.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Passkey Removal Signs Mutable Index

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AuthFacet.sol#L370
REVIEW	Main Review

DESCRIPTION

`initiateRemovePasskey` authorizes passkey removal by signing only the passkey array index, instead of signing a stable identifier of the target passkey. This is unsafe because the passkey array is mutable, so the passkey stored at a given index can change between the moment the signature is created and the moment the removal is initiated.

For example, assume the wallet passkeys are [A, B, C], and a signer creates a signature to remove the passkey at index 1, intending to remove B. Before that request is submitted, another signer removes B or otherwise changes the array ordering. As a result, index 1 may now point to a different passkey. The old signature will still be valid because it only commits to the numeric index, but it may now remove a different passkey than the signer originally intended.

RECOMMENDATION

Bind passkey removal signatures to an immutable identifier of the target passkey rather than its current position in the array.

RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

Stale Timestamps After Direct Mode Transitions

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibDirectMode.sol#L240-L254>

REVIEW Main Review

DESCRIPTION

`getPendingDirectMode` and `getPendingExitDirectMode` both document that `activatesAt` should return 0 when nothing is pending. However, after direct mode activates, `getPendingDirectMode` still returns the historical `directModeAt` timestamp instead of 0. The same applies to `getPendingExitDirectMode`, after an exit completes, it returns the stale `exitDirectModeAt` value. In both cases `isPending` correctly returns false, but `activatesAt` is non-zero despite nothing being pending, contradicting the documented behavior.

In addition, after a complete enter→exit cycle, `directModeAt` is still non-zero because `initiateExitDirectMode` never clears it. This is intentional to allow the cooldown process to take place. However, this causes `getPendingDirectMode` to return a stale `activatesAt` value even though direct mode is no longer active, which could mislead users/integrators into misinterpreting the wallet's direct mode state.

RECOMMENDATION

Return 0 for `activatesAt` when nothing is pending for both `getPendingDirectMode` and `getPendingExitDirectMode`:

```
function getPendingDirectMode() internal view returns (uint64 activatesAt, bool isPending) {
    uint64 timestamp = directModeStorage().directModeAt;
    isPending = timestamp > 0 && block.timestamp < timestamp;
    return (isPending ? timestamp : 0, isPending);
}
```

RESOLUTION

Status: Resolved.

LOW **L-04**

RESOLVED

Direct Mode Cancel Lacks State Binding

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/DirectModeFacet.sol#L106 https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/DirectModeFacet.sol#L148
REVIEW	Main Review

DESCRIPTION

The direct mode cancel messages are not tied to a specific pending `directModeAt` or `exitDirectModeAt` value. Both cancel payloads sign only a bare nonce and deadline. When a cancel signature is later submitted, the contract only checks that some pending action currently exists. It does not verify that the signature was created for the current pending timestamp:

Because of this, an old unused cancel signature can be replayed against a different future direct-mode action, as long as the signer's nonce is still unchanged and the deadline has not expired. For example, a signer may sign a cancellation message for one pending direct-mode entry, but if it is not submitted at that time, it can remain valid and later be used against a different pending direct-mode action. If a new entry or exit is initiated before that signer uses the same nonce for another action, the stale signature can still be submitted and will cancel the new pending state even though it was not created for that specific case.

RECOMMENDATION

Consider including the current `directModeAt` and `exitDirectModeAt` value in the signed cancel payload so each cancel signature is tied to a specific pending action.

RESOLUTION

Status: Resolved.

LOW **L-05**

ACKNOWLEDGED

Wallet recovery jeopardized by factory owner

CATEGORY	Trust Assumptions
LOCATION	https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAuth.sol#L1243
REVIEW	Main Review

DESCRIPTION

`RecoveryFacet` grants the ability to add a new signer to the recovery owners. A recovery is initiated and once the 10-days waiting period passes, it can be completed by calling `executeRecovery()`. If the signer to be added is `E0A/smart contract`, the code proceeds executing `addOwnerDirect()`.

There exists the check that reverts the transaction if the owner is the same as the executor in the factory.

```
// Re-check executor at execution time (executor may have changed since recovery initiation)
address currentExecutor = IUserProxyFactory(LibFactory.getFactory()).executor();
if (_owner == currentExecutor) revert ExecutorCannotBeOwner();
```

This makes the wallet dependent on the factory, even in direct mode. A compromised factory owner can intercept every recovery request and change the executor to the owner that's being added, rendering the recovery mechanism useless for that wallet, unless a passkey is being added.

RECOMMENDATION

Either remove the validation check and allow even the executor to be added or clearly document the risk and inform users that they must use passkeys in this case.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

No path back to default recovery admins

CATEGORY Configuration
LOCATION src/facets/RecoveryFacet.sol
REVIEW Main Review

DESCRIPTION

Once a wallet sets a custom recovery admin list via `setRecoveryConfig()`, `rs.hasCustomConfig` is set to true and there is no supported way to revert to factory default recovery admins while staying opted-in. The only available option is `optOutRecovery()`, which disables recovery entirely. This prevents wallets from adopting future factory default changes.

RECOMMENDATION

Allow reverting to defaults by accepting `admins.length == 0` in `setRecoveryConfig()` as a "use factory defaults" signal, clearing `customRecoveryAdmins` and setting `hasCustomConfig = false`.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Inconsistent function naming

CATEGORY	Best Practices
LOCATION	src/libraries/LibAuth.sol
REVIEW	Main Review

DESCRIPTION

There is inconsistency between the naming of the functions that perform actions related to owner addition vs the functions performing actions related to passkey addition in `LibAuth`:

- `initiateAddOwner()` vs `initiatePendingPasskey()`
- `executeAddOwner()` vs `executePendingPasskey()`
- `cancelAddOwner()` vs `cancelPendingPasskey()`

Compared to the functions for removal, where the naming is consistent:

- `initiateRemoveOwner()/initiateRemovePasskey()`
- `executeRemoveOwner()/executeRemovePasskey()`
- `cancelRemoveOwner()/cancelRemovePasskey()`

RECOMMENDATION

Consider making the naming consistent for add/remove.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

REENTRANCY_SLOT deviates from ERC-7201

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibReentrancyGuard.sol#L23>

REVIEW Main Review

DESCRIPTION

The comment in `LibReentrancyGuard` says:

- * The slot is derived using the ERC-7201 namespaced pattern:
- * `keccak256("ethenapay.storage.ReentrancyGuard") - 1`, masked to align.

However, the ERC-7201 formula (<https://eips.ethereum.org/EIPS/eip-7201#formula>) hashes the `id` of the namespace twice before applying `&` to it.

```
keccak256(abi.encode(uint256(keccak256(bytes(id))) - 1)) & ~bytes32(uint256(0xff))
```

The `REENTRANCY_SLOT` is hashed only once and therefore it deviates from the actual `ERC-7201` standard.

RECOMMENDATION

Consider hashing the namespace id twice.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

RESOLVED

Redundant exists checks before event emission

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AuthFacet.sol#L329>, <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AuthFacet.sol#L481>, <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AuthFacet.sol#L555>

REVIEW Main Review

DESCRIPTION

In `approveAddPasskey()`, `approveAddOwner()`, and `approveRemoveOwner()`, the `exists` flag read from `LibAuth.getPendingPasskey()`, `LibAuth.getPendingOwner()`, and `LibAuth.getPendingOwnerRemoval()` is used to gate `PasskeyAdditionExecuted`, `OwnerAdditionExecuted`, and `OwnerRemovalExecuted`. However, `LibAuth.executePendingPasskey()`, `LibAuth.executeAddOwner()`, and `LibAuth.executeRemoveOwner()` all revert if the pending entry does not exist or is not ready, so reaching the emit implies `exists` was `true`. The conditional check adds minor gas and noise without changing behavior.

RECOMMENDATION

Remove the `if (exists)` guards and always emit `PasskeyAdditionExecuted`, `OwnerAdditionExecuted`, and `OwnerRemovalExecuted` after the respective `LibAuth.execute*()` call succeeds.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-04

RESOLVED

Wallet deploy event omits passkey data

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/UserProxyFactoryBeacon.sol#L103>

REVIEW Main Review

DESCRIPTION

`deployWalletWithBoth()` accepts passkey coordinates (`qx`, `qy`) but the corresponding `WalletDeployedWithBoth` event does not emit them. This makes off-chain indexing and monitoring less convenient compared to `WalletDeployedWithPasskey`, which includes `qx` and `qy`. Consumers must parse calldata or query the deployed wallet state to recover the passkey data.

RECOMMENDATION

Include `qx` and `qy` in the `WalletDeployedWithBoth` event and emit them from `deployWalletWithBoth()` so the event is self-contained for indexers.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

ACKNOWLEDGED

No Per-Token Zero-Spend limit

CATEGORY Suggestion

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAllowance.sol#L74-L77>

REVIEW Main Review

DESCRIPTION

`LibAllowance` uses 0 (default value) to allow unlimited spending. Since there is no value that disallows the executor from spending a specific token, users who want to block executor access to a specific token must enter direct mode, which disables executor operations across all tokens. This means a user cannot block executor spending for one token while keeping daily limits active for others. Setting the token limit to `1 wei` still technically allows spending, since it is not truly zero.

RECOMMENDATION

Reserve `type(uint256).max` for unlimited and treat 0 as "no spending permitted". Alternatively, consider adding a per-token bool disabled flag to the `AllowanceStorage` struct.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

RESOLVED

Passkey Removal Event Omits Key Data

CATEGORY Events

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AuthFacet.sol#L58>

REVIEW Main Review

DESCRIPTION

`PasskeyRemovalInitiated` emits only the original passkey index, while the pending removal state stores both the index and the passkey coordinates. This makes the event inconsistent with

`PasskeyAdditionInitiated`, which emits `qx` and `qy` directly.

Passkey removal is based on `qx` and `qy`, not the original index. Consequently, the event alone does not clearly identify which passkey is being removed, and off-chain consumers cannot determine it without querying contract state.

RECOMMENDATION

Emit `qx` and `qy` in `PasskeyRemovalInitiated` so the event fully identifies the passkey being removed.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-07

RESOLVED

Default recovery admins cannot be empty

CATEGORY Configuration

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/UserProxyFactoryBeacon.sol#L458>

REVIEW Main Review

DESCRIPTION

`UserProxyFactoryBeacon.scheduleDefaultRecoveryAdmins()` reverts when `admins.length == 0` and `LibRecovery.validateAdmins()` rejects `address(0)`, so the factory default recovery admin list cannot be cleared to an empty set. This means removing defaults is only possible per wallet via `optOutRecovery()` rather than at the factory level.

RECOMMENDATION

If clearing factory defaults is intended, allow `admins.length == 0` in `scheduleDefaultRecoveryAdmins()` to explicitly clear `_defaultRecoveryAdmins`, or add a dedicated `clearDefaultRecoveryAdmins()` flow. Otherwise, document that defaults are always non-empty and that wallets must use `optOutRecovery()` to disable recovery.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-08

RESOLVED

Recovery Config Events Lack Admin Data

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/RecoveryFacet.sol#L65-L68 https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/UserProxyFactoryBeacon.sol#L120-L121
REVIEW	Main Review

DESCRIPTION

`RecoveryConfigChanged` emits only `adminCount`, even though `setRecoveryConfig` applies a specific recovery admin set. A similar issue exists in the factory, where `DefaultRecoveryAdminsScheduled` and `DefaultRecoveryAdminsExecuted` emit only the number of admins rather than the actual admin array or its hash. As a result, an off-chain consumers cannot determine the exact recovery admin configuration from logs alone.

RECOMMENDATION

Consider emitting the full admin array in the event, or at least emit a hash of it, so off-chain consumers can easily identify which recovery admin set was configured from the logs.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-09

ACKNOWLEDGED

Unlimited mode skips daily spend accounting

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAllowance.sol#L95>

REVIEW Main Review

DESCRIPTION

When the configured `dailyLimit()` is `0` (unlimited), `_checkAndUpdateDailyLimit()` returns early and does not update `dailySpending`. Spends via `spendDailyAllowance()` and `spendDailyAllowanceEth()` still emit `AllowanceSpent()`, so offchain consumers cannot distinguish whether a spend was accounted toward a daily cap, while `dailySpent()` / `dailyRemaining()` stay unchanged. If a user later calls `instantSetAllowance()` to set a finite `dailyLimit()` the same day, the executor can spend the full new limit on top of earlier unlimited spends.

RECOMMENDATION

If the intended semantics are "same-day caps apply to all spending," update `_checkAndUpdateDailyLimit()` to update `dailySpending` even when `dailyLimit()` is `0` (skip only the revert). If unlimited spending should remain unaccounted, emit a distinct event or extend `AllowanceSpent()` with an `accounted` flag or `limit` field so offchain systems can disambiguate.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

RESOLVED

Natspec says wallet nonce for allowance

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAllowance.sol#L134>

REVIEW Main Review

DESCRIPTION

The NatSpec for `hashInstantSetAllowance()` says `nonce` is the "current wallet nonce," but the code consumes a per-signer nonce via `LibAuth._useCheckedNonce()` in `instantSetAllowance()`. There is no global wallet nonce; nonces are tracked per signer across the system. This mismatch can mislead integrators.

RECOMMENDATION

Update the NatSpec to say the `nonce` is the current per-signer nonce for replay protection, consistent with the rest of the codebase/docs.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Max amount transfer handling

CATEGORY	Validation
LOCATION	WithdrawalFacet.sol
REVIEW	Main Review

DESCRIPTION

`AllowanceFacet.spendDailyAllowance()` rejects `amount == type(uint256).max` because it compares amount to `balanceOf()` before transfer. There are some ERC20 tokens that accept `uint256.max` as "transfer the available balance".

There is no balance check in `WithdrawalFacet.executorTokenWithdrawal()` and `WithdrawalFacet.directTokenWithdrawal()` - they pass `amount` directly to `safeTransfer()` and emit `ExecutorWithdrawal/DirectWithdrawal` with the same amount. If this feature is used, the event will emit `uint256.max`, while the transfer value will be different.

RECOMMENDATION

Consider adding the balance check to `WithdrawalFacet` as well.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-12

RESOLVED

Executor Address Can Be Set As Recovery Admin

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibRecovery.sol#L165-L177>

REVIEW Main Review

DESCRIPTION

When adding a recovery admin via `setRecoveryConfig` or the factory's `scheduleDefaultRecoveryAdmins`, `LibRecovery.validateAdmins` checks for zero addresses and duplicates but does not check whether any admin is the current executor. The protocol prevents the executor from being an owner, but does not apply the same restriction to recovery admin configuration, which would give the executor more privilege than what is intended for that role.

RECOMMENDATION

Check against `factory.executor()` in `validateAdmins`.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-13**

ACKNOWLEDGED

Allowance Set During Pause Cannot Be Spent

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AllowanceFacet.sol#L85>

REVIEW Main Review

DESCRIPTION

`instantSetAllowance` uses `enforceUserNotBlocked`, which permits execution when the user is in `direct mode` even during a global pause. However, `spendDailyAllowance` and `spendDailyAllowanceEth` use `enforceExecutorNotBlocked`, which blocks spending during both global pause and direct mode. This means a user can change daily limits during a global pause via direct mode, but the executor cannot spend against those limits until both the pause lifts and direct mode is exited. This could mislead integrators into thinking the allowance is immediately in effect.

RECOMMENDATION

Document that allowances set during a global pause via direct mode only take effect for executor spending once the pause is lifted and direct mode is exited.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Settlement Flows Lack Batch Execution

CATEGORY	Suggestion
LOCATION	global
REVIEW	Main Review

— DESCRIPTION

The protocol intends to support delayed settlement of user obligations, for example when the executor later deducts configured allowance for activity. However, the current settlement and allowance-spend flows are exposed only as single operations. If multiple obligations, or multiple user actions and their related obligations, must be settled on-chain, the executor has to submit them one by one.

This increases gas overhead and operational friction compared to a batch-based design. At scale, the executor may be forced to submit a large number of separate transactions for actions that could otherwise be grouped together.

— RECOMMENDATION

Consider adding support for batching transfer and allowance-spend operations, so multiple off-chain computed obligations can be settled in a single transaction when appropriate.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

ACKNOWLEDGED

Executor settlement depends on chain state

CATEGORY Trust Assumptions

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AllowanceFacet.sol#L108>

REVIEW Main Review

DESCRIPTION

Executor settlement relies on `spendDailyAllowance()` / `spendDailyAllowanceEth()` executing after an offchain service is delivered. Even with a "smart executor" that tracks balances and state, several on-chain conditions can still change between service delivery, transaction submission, and inclusion, causing executor pulls to revert. Examples include: direct mode activation (`initiateDirectMode()` → `isDirectMode()` becomes true) which makes `LibDirectMode.enforceExecutorNotBlocked()` revert; global pause (`globalPaused()`) which also blocks executor pulls; token behaviors like rebasing, fee-on-transfer, blacklists, or pausable hooks that can change `IERC20.balanceOf()` or make `safeTransfer()` revert after a pre-check; and reorgs/ordering delays that invalidate recent state observations. These are inherent settlement/credit risks rather than protocol bugs, but they can cause valid executor pulls to fail even when allowances are unlimited.

RECOMMENDATION

Document this as an explicit trust/operational assumption and implement mitigations on the executor side: minimize time between service and `spendDailyAllowance()` / `spendDailyAllowanceEth()` submission, monitor `pendingDirectMode()` / `isDirectMode()` and `globalPaused()`, use token allowlists or token-specific handling for rebasing/fee-on-transfer tokens, and consider prepayment/escrow or atomic "set + spend" flows for high-value services.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-16

ACKNOWLEDGED

ETH naming for Avalanche deployment

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/AllowanceFacet.sol#L44>

REVIEW Main Review

DESCRIPTION

The codebase uses ETH-specific naming (for example `ETH_TOKEN`, `UseSpendAllowanceEthForETH()`, and `spendDailyAllowanceEth()`), and comments like `dailyLimit()` mention "ETH." On Avalanche, this can confuse integrators and users, since the native token is not ETH. The functionality uses `address(0)` as the native token, but the naming implies Ethereum-specific semantics.

RECOMMENDATION

Update comments, errors, and public-facing terminology to say "native token" (or chain-specific token like AVAX) instead of ETH, or document clearly that "ETH" is used as a generic alias for the native token across deployments.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-17

ACKNOWLEDGED

Pending Executor Can Be Added As Wallet Owner

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAuth.sol#L673-L677>

REVIEW Main Review

DESCRIPTION

`executeAddOwner` and `addOwnerDirect` check `pending.owner == factory.executor()` to enforce the invariant that the executor cannot also be an owner. However, neither checks against `factory.pendingExecutor()`. A wallet owner can call `initiateAddOwner` with the pending executor address during the 48h executor change delay. Since the address is not yet the active executor, it passes all checks. After the 24h signer cooldown, `executeAddOwner` re-checks against the current executor, which is still the old one. The owner addition succeeds. When the executor change finalizes at 48h, the address holds both the owner and executor roles, breaking the invariant.

Note that adding a `pendingExecutor` check on the wallet side would introduce a DoS vector: a compromised factory owner could block any wallet's owner additions by scheduling a pending executor change to match that wallet's pending owner address. Since there is no delay, it can immediately DoS the pending owner addition execution.

RECOMMENDATION

Document the ability for users to add a pending executor as an owner as a known constraint.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-18

ACKNOWLEDGED

Facet Upgrade Can Break Pending Recovery

CATEGORY Upgradeability

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/UserProxyFactoryBeacon.sol#L299>

REVIEW Main Review

DESCRIPTION

Recovery has a fixed 10-day timelock while facet updates have a 48h timelock. If a facet update that modifies or removes recovery-related selectors is executed during an active recovery period, `executeRecovery` will revert because the selector no longer resolves to a valid facet. The recovery must then be re-initiated after the facet is restored, adding at least another 10 days. Both `scheduleFacetUpdate` and `executeFacetUpdate` have `onlyOwner` modifier, so this is not an external attack vector, although it can pose as an operational risk during upgrades.

RECOMMENDATION

Document that pending recoveries should be checked before executing facet updates that affect recovery selectors. Alternatively, gate facet updates behind a check for active recoveries across affected wallets.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-19

RESOLVED

Recovery Admin Validation Copies Calldata

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/facets/RecoveryFacet.sol#L139 https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/UserProxyFactoryBeacon.sol#L455 https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibRecovery.sol#L167
REVIEW	Main Review

DESCRIPTION

`setRecoveryConfig` and `scheduleDefaultRecoveryAdmins` function accept the recovery admin array as calldata, but both pass it to `LibRecovery.validateAdmins`, which takes `address[] memory`. As a result, Solidity copies the full array from calldata into memory before validation, even though the helper only reads the array and does not modify it. This introduces unnecessary gas overhead on every recovery config update and default admin scheduling call.

RECOMMENDATION

Change `LibRecovery.validateAdmins` to accept `address[] calldata`.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-20

RESOLVED

Executor Verification Copies Calldata To Memory

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAuth.sol#L384>

REVIEW Main Review

DESCRIPTION

`verifyExecutorSignature` receives `executorSig` as `bytes calldata` but passes it to `SignatureChecker.isValidSignatureNow` which takes `bytes memory`, forcing an unnecessary `calldata→memory` copy for the signature on every call if the executor is an EOA. OpenZeppelin `SignatureChecker.sol` provides `isValidSignatureNowCalldata` which uses `tryRecoverCalldata` on the EOA path, avoiding the copy and saving gas. There is no difference if the executor is not an EOA, since the `ERC-1271` path will always require `calldata→memory` conversion due to the external call, however in the case the executor is an EOA, the conversion is unnecessary and wastes gas.

RECOMMENDATION

Replace `isValidSignatureNow` with `isValidSignatureNowCalldata` in `verifyExecutorSignature`.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-21

ACKNOWLEDGED

Ownership Transfer Lacks Timelock

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/UserProxyFactoryBeacon.sol#L28>

REVIEW Main Review

DESCRIPTION

All privileged factory operations use a 48h timelock: beacon upgrades, facet updates, executor changes, and recovery admin changes. However, ownership transfer via `Ownable2Step` has no delay, the pending owner can accept the ownership transfer immediately. An ownership transfer to a compromised or incorrect address takes effect instantly with no monitoring window for users to react.

RECOMMENDATION

Consider adding a timelock to ownership transfer consistent with other admin operations, or document that ownership transfer is immediate (no delay between nomination and acceptance) and that users should monitor `OwnershipTransferStarted` events.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-22

RESOLVED

Natspec omits self-address check

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/ethenapay-wallet-contracts-team1-1772743322801/blob/9122473f53100151858cf709f91062d63aa82f8a/src/libraries/LibAuth.sol#L1238>

REVIEW Main Review

DESCRIPTION

The NatSpec for `addOwnerDirect()` says it reverts with `InvalidOwnerAddress()` if the address is 0, but the implementation also reverts with the same error when `_owner == address(this)`. This mismatch can confuse integrators about the actual validation rules.

RECOMMENDATION

Update the NatSpec to mention both conditions (`address(0)` and `address(this)`)

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-23**

ACKNOWLEDGED

Skipped WebAuthn verification steps

CATEGORY	Warning
LOCATION	LibAuth
REVIEW	Main Review

— DESCRIPTION

As described on the OZ website (<https://docs.openzeppelin.com/community-contracts/api/utls/cryptography?#WebAuthn>), origin and rpId checks, as well as signature counter, extension outputs and attestation checks, are not performed.

It's assumed the authenticator will take care of the origin check. This increases the risk of phishing attacks if the authenticator fails to validate the origin.

— RECOMMENDATION

Recheck if skipping each of the validations is fine.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Default recovery path doesn't modify optedOut

CATEGORY Unexpected Behavior

LOCATION <https://github.com/chainnodesorg/ethenapay-wallet-contracts/blob/bd1cf64454ba4eb01755402086c27fe6f3e7e7a2/src/facets/RecoveryFacet.sol#L161-L173>

REVIEW Remediation Review

DESCRIPTION

In `RecoveryFacet.setRecoveryConfig()`, when called with an empty admins array (`admins.length == 0`), the function resets `hasCustomConfig` to false and deletes `customRecoveryAdmins`, but fails to set `rs.optedOut = false`.

A wallet owner who has previously opted out of recovery and later calls `setRecoveryConfig()` with an empty admins array — intending to revert to factory default recovery — will believe recovery is re-enabled. However, `optedOut` remains true, causing `initiateRecovery()` to revert via `enforceNotOptedOut()`.

If the owner subsequently loses all signing keys, the wallet becomes permanently unrecoverable. No recovery admin can initiate recovery, and the owner has no keys to correct the misconfigured state. All funds in the wallet are locked forever.

RECOMMENDATION

Set `optedOut = false` when reverting back to default recovery admins.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Lack of signer parameter documentation

CATEGORY Documentation
LOCATION AuthFacet.sol
REVIEW Remediation Review

DESCRIPTION

The new `signer` parameter added to the functions in `AuthFacet` is not documented in the NatSpecs.

RECOMMENDATION

Add `@param signer` documentation to the NatSpec of each function.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.